

Teste de Software

ESCOLA ESTADUAL DE ENSINO PROFISSIONAL
Prof. DAVI MAGALHÃES
QUALIDADE E TESTE DE SOFTWARE
AULA 04

Teste de Software

Teste de software é um processo fundamental no desenvolvimento de sistemas que visa **avaliar e validar** se um software **atende aos requisitos especificados** e funciona corretamente em diferentes cenários. O objetivo principal dos testes é identificar defeitos (bugs), melhorar a qualidade do software e garantir que ele seja **confiável antes de ser entregue** aos usuários finais.



Tipos de Testes

Os testes podem ser classificados de várias formas, dependendo do objetivo e do nível de detalhamento. Algumas categorias comuns incluem:



Tipos de Testes

Os testes podem ser classificados de várias formas, dependendo do objetivo e do nível de detalhamento. Algumas categorias comuns incluem:

Testes Funcionais:

- Avaliam se o software está funcionando conforme os requisitos especificados.



Tipos de Testes

Os testes podem ser classificados de várias formas, dependendo do objetivo e do nível de detalhamento. Algumas categorias comuns incluem:

Testes Funcionais:

- Avaliam se o software está funcionando conforme os requisitos especificados.

Testes Não Funcionais:

- Analisam características como desempenho, segurança e usabilidade.
- 

Testes Funcionais

Avaliam se o software está funcionando conforme os requisitos especificados.

Exemplos:

1. **Testes de Unidade:** verificam partes isoladas do código, geralmente funções ou métodos.



Testes Funcionais

Avaliam se o software está funcionando conforme os requisitos especificados.

Exemplos:

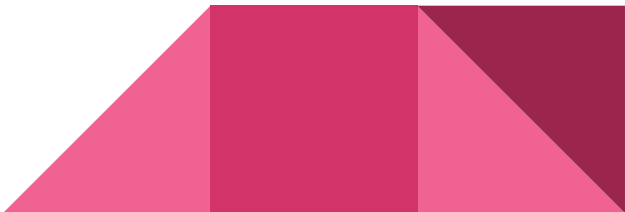
1. **Testes de Unidade:** verificam partes isoladas do código, geralmente funções ou métodos.
2. **Testes de Integração:** verificam se módulos ou componentes do sistema funcionam corretamente em conjunto.



Testes Funcionais

Avaliam se o software está funcionando conforme os requisitos especificados.

Exemplos:

1. **Testes de Unidade:** verificam partes isoladas do código, geralmente funções ou métodos.
 2. **Testes de Integração:** verificam se módulos ou componentes do sistema funcionam corretamente em conjunto.
 3. **Testes de Sistema:** avaliam o sistema como um todo, verificando seu comportamento geral.
- 

Testes Funcionais

Avaliam se o software está funcionando conforme os requisitos especificados.

Exemplos:

1. **Testes de Unidade:** verificam partes isoladas do código, geralmente funções ou métodos.
 2. **Testes de Integração:** verificam se módulos ou componentes do sistema funcionam corretamente em conjunto.
 3. **Testes de Sistema:** avaliam o sistema como um todo, verificando seu comportamento geral.
 4. **Testes de Aceitação:** garantem que o software atende às expectativas do cliente ou usuário final.
- 

Testes Não Funcionais

Analisam características como desempenho, segurança e usabilidade. Exemplos:

1. **Testes de Desempenho:** verificam o tempo de resposta e a escalabilidade do software.



Testes Não Funcionais

Analisam características como desempenho, segurança e usabilidade. Exemplos:

1. **Testes de Desempenho:** verificam o tempo de resposta e a escalabilidade do software.
2. **Testes de Segurança:** identificam vulnerabilidades no sistema.



Testes Não Funcionais

Analisam características como desempenho, segurança e usabilidade. Exemplos:

1. **Testes de Desempenho:** verificam o tempo de resposta e a escalabilidade do software.
2. **Testes de Segurança:** identificam vulnerabilidades no sistema.
3. **Testes de Usabilidade:** analisam a experiência do usuário.



Importância dos Testes

- Reduz custos de manutenção ao detectar bugs mais cedo.



Importância dos Testes

- Reduz custos de manutenção ao detectar bugs mais cedo.
- Garante a segurança e estabilidade do sistema.



Importância dos Testes

- Reduz custos de manutenção ao detectar bugs mais cedo.
- Garante a segurança e estabilidade do sistema.
- Melhora a experiência do usuário.



Importância dos Testes

- Reduz custos de manutenção ao detectar bugs mais cedo.
- Garante a segurança e estabilidade do sistema.
- Melhora a experiência do usuário.
- Aumenta a confiabilidade e a reputação do software no mercado.



Objetivos do Teste de Software

Os testes de software têm como principal finalidade garantir a qualidade do sistema, prevenindo falhas e assegurando que o software atenda aos requisitos esperados. Para isso, os testes possuem diversos objetivos específicos, que podem variar de acordo com o contexto do projeto.



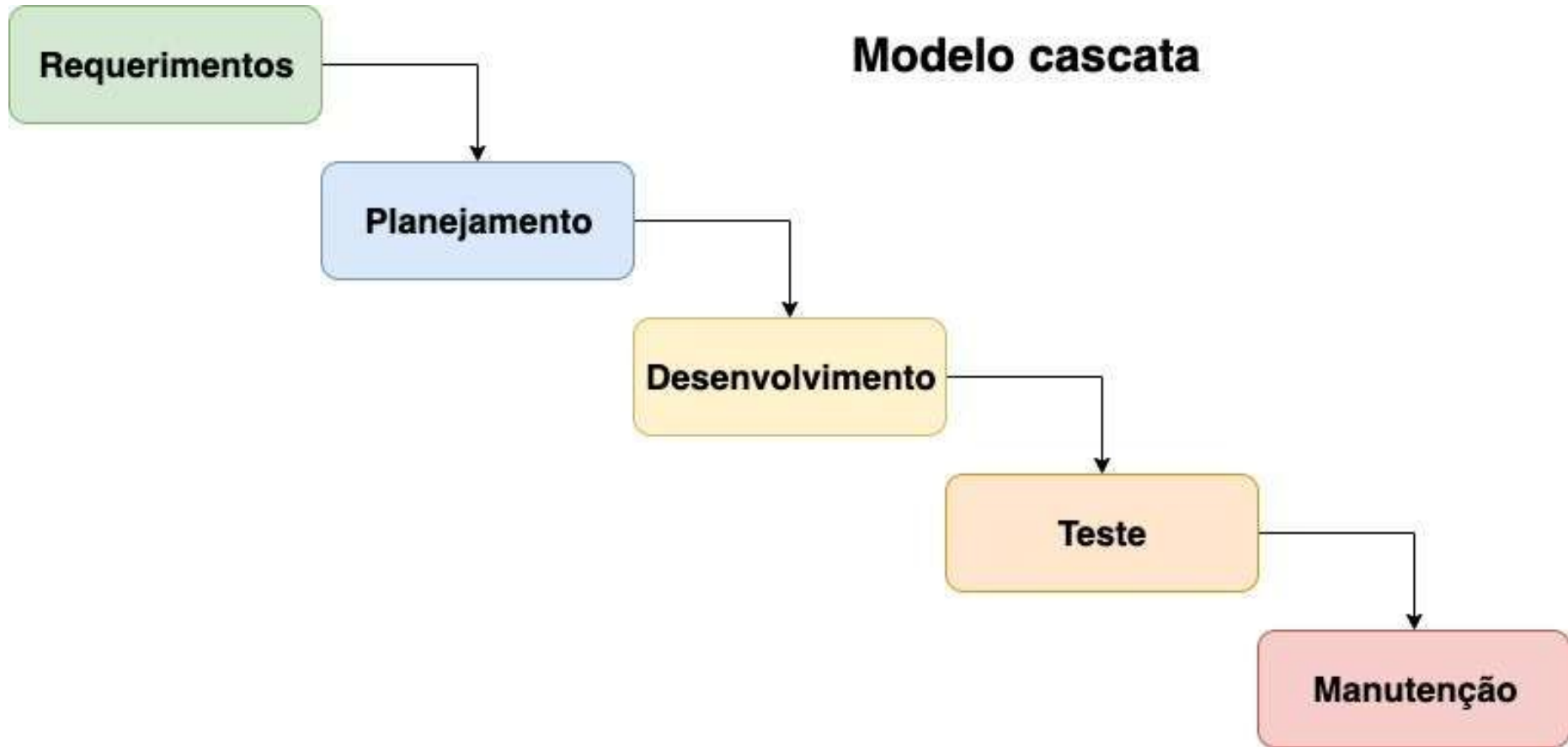
Objetivos do Teste de Software

Detectar Defeitos (Bugs) o Mais Cedo Possível

Os testes são projetados para identificar erros no código antes que eles causem problemas em produção. Quanto mais cedo um defeito é encontrado, menor é o custo para corrigi-lo.



Modelo cascata



Objetivos do Teste de Software

Garantir que o Software Atenda aos Requisitos

Os testes verificam se o software está em conformidade com as especificações funcionais e não funcionais definidas no projeto. Isso evita que funcionalidades importantes sejam esquecidas ou mal implementadas.



Objetivos do Teste de Software

Prevenir Falhas em Produção

Um dos principais objetivos dos testes é evitar que problemas cheguem ao usuário final. Isso reduz impactos negativos, como insatisfação do cliente, perda de receita ou falhas críticas em sistemas sensíveis.



Objetivos do Teste de Software

Melhorar a Qualidade do Software

Testes bem planejados ajudam a garantir que o software seja confiável, seguro, eficiente e de fácil usabilidade. Isso melhora a experiência do usuário e reduz a necessidade de retrabalho.



Objetivos do Teste de Software

Garantir a Segurança do Sistema

Os testes de segurança buscam identificar vulnerabilidades, como falhas de autenticação, exposição de dados sensíveis e ataques cibernéticos. Esse objetivo é essencial para sistemas que lidam com informações críticas.



Objetivos do Teste de Software

Validar a Performance e Escalabilidade

Os testes de desempenho analisam a capacidade do software de lidar com cargas de trabalho variadas, verificando tempos de resposta e consumo de recursos. Isso garante que o sistema funcione bem sob diferentes condições.



Objetivos do Teste de Software

Apoiar a Manutenção e Evolução do Software

Testes automatizados e bem documentados facilitam futuras modificações no código, permitindo que novas funcionalidades sejam adicionadas sem comprometer o funcionamento do sistema.



Objetivos do Teste de Software

Assegurar a Confiabilidade e a Estabilidade

Os testes contínuos garantem que o software se comporte de forma previsível e estável, reduzindo falhas inesperadas.



Objetivos do Teste de Software

Os testes de software não são apenas uma etapa do desenvolvimento, mas um **processo contínuo** para assegurar a qualidade e confiabilidade do sistema. Ao investir em testes eficazes, as empresas **reduzem custos, evitam problemas e entregam produtos melhores** para seus usuários.



Ciclo de Vida do Teste

O Ciclo de Vida do Teste de Software (STLC - Software Testing Life Cycle) é um **conjunto de etapas sistemáticas** que garantem que os testes sejam bem planejados, executados e monitorados.



Ciclo de Vida do Teste

O Ciclo de Vida do Teste de Software (STLC - Software Testing Life Cycle) é um conjunto de etapas sistemáticas que garantem que os testes sejam bem planejados, executados e monitorados. Assim como o desenvolvimento de software segue um ciclo de vida (SDLC - Software Development Life Cycle), os testes também possuem um processo estruturado para assegurar a qualidade do software.

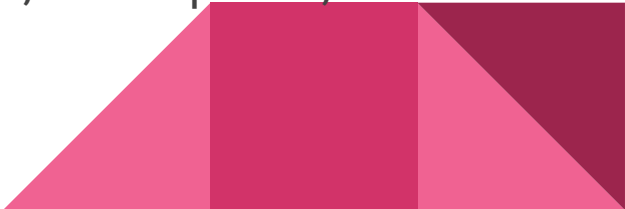


Ciclo de Vida do Teste: FASES

1. Análise de Requisitos

Objetivo: Compreender os requisitos do software para definir a estratégia de testes.

Atividades:

- Análise dos requisitos funcionais e não funcionais.
 - Identificação de ambiguidades ou inconsistências.
 - Definição dos tipos de testes necessários (funcionais, desempenho, segurança etc.).
- 

Ciclo de Vida do Teste: FASES

2. Planejamento de Testes

Objetivo: Definir o escopo, estratégia e cronograma dos testes.

Atividades:

- Escolha das ferramentas e técnicas de teste.
- Estimativa de tempo e recursos necessários.
- Definição dos critérios de entrada e saída para os testes.

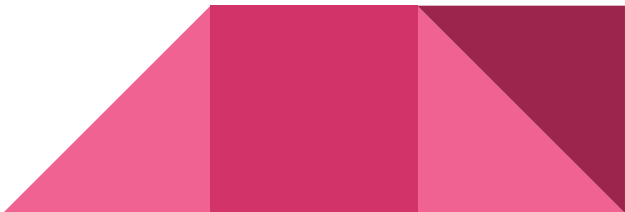


Ciclo de Vida do Teste: FASES

3. Projeto dos Casos de Teste

Objetivo: Criar os cenários e casos de teste com base nos requisitos analisados.

Atividades:

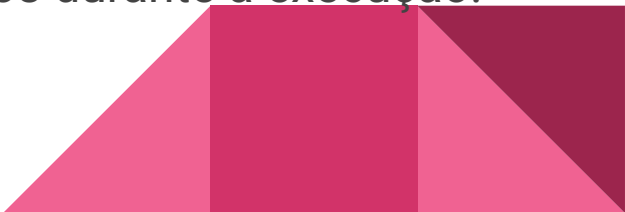
- Criação de casos de teste detalhados (entrada, ações e resultados esperados).
 - Revisão e validação dos casos de teste.
 - Preparação dos dados de teste (se necessário).
- 

Ciclo de Vida do Teste: FASES

4. Configuração do Ambiente de Testes

Objetivo: Preparar a infraestrutura necessária para a execução dos testes.

Atividades:

- Instalação e configuração do software, banco de dados e ferramentas.
 - Garantia de que o ambiente de teste seja o mais próximo possível do ambiente real.
 - Validação do ambiente para evitar problemas técnicos durante a execução.
- 

Ciclo de Vida do Teste: FASES

5. Execução dos Testes

Objetivo: Rodar os testes e verificar se o sistema funciona conforme esperado.

Atividades:

- Execução dos casos de teste manuais ou automatizados.
- Registro dos resultados (sucesso ou falha).
- Reporte de defeitos e reexecução dos testes após correção.

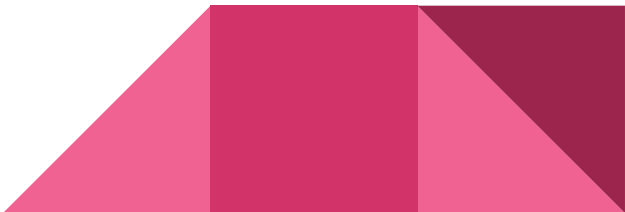


Ciclo de Vida do Teste: FASES

6. Encerramento e Relatório de Testes

Objetivo: Avaliar os resultados dos testes e validar se os critérios foram atendidos.

Atividades:

- Comparação dos resultados obtidos com os esperados.
 - Geração de relatórios com métricas e insights.
 - Tomada de decisão sobre a liberação do software.
- 

Ciclo de Vida do Teste

O ciclo de vida do teste de software ajuda a garantir que os testes sejam **organizados, eficientes e eficazes**. Seguir essas etapas reduz riscos e melhora a qualidade do software antes do lançamento.



Tipos de Testes

Testes Funcionais

Os testes funcionais são um tipo de teste de software que verifica se o sistema atende aos **requisitos funcionais especificados**. Eles avaliam o comportamento do software ao testar suas funcionalidades com diferentes entradas e validando se os resultados obtidos estão corretos.



Testes Funcionais

Os testes funcionais são um tipo de teste de software que verifica se o sistema atende aos **requisitos funcionais especificados**. Eles avaliam o comportamento do software ao testar suas funcionalidades com diferentes entradas e validando se os resultados obtidos estão corretos.

Esse tipo de teste é essencial para garantir que as principais operações do sistema, como login, cadastro, processamento de dados e exibição de informações, funcionem conforme esperado.



Testes Funcionais (Requisito Funcional)

Um requisito funcional descreve o **comportamento e as funcionalidades** que um sistema deve ter para atender às necessidades dos usuários. Ele especifica o que o sistema deve fazer, sem detalhar como isso será implementado.



Testes Funcionais (Requisito Funcional)

Características dos Requisitos Funcionais

- Definem ações, processos e regras do sistema.
- São baseados nas necessidades do usuário ou cliente.
- Devem ser claros, objetivos e testáveis.



Testes Funcionais (Requisito Funcional)

Exemplos de Requisitos Funcionais

- O sistema deve permitir que os usuários realizem login com e-mail e senha.
- O software deve gerar relatórios mensais de vendas em PDF.
- A plataforma deve enviar notificações por e-mail ao confirmar uma compra.



Testes Funcionais (Requisito Funcional)

Diferença entre Requisito Funcional e Não Funcional

- **Funcional:** **O que** o sistema faz (Ex.: "Permitir cadastro de usuários").
- **Não Funcional:** **Como** o sistema deve se comportar (Ex.: "O tempo de resposta do cadastro deve ser menor que 2 segundos").



Testes Funcionais: **Teste de Unidade**

Testam **partes individuais do código** (como funções, métodos e classes) de forma isolada. São normalmente automatizados e executados por desenvolvedores.

Exemplo: Testar se uma função de cálculo de média retorna o valor correto.



Testes Funcionais: **Teste de Integração**

Verificam a **comunicação entre diferentes módulos** do sistema, garantindo que funcionem corretamente juntos.

Exemplo: Verificar se o módulo de pagamento se comunica corretamente com o módulo de pedidos.



Testes Funcionais: **Teste de Sistema**

Avaliam o software **como um todo** para garantir que todas as funcionalidades funcionem bem juntas.

Exemplo: Testar se um usuário consegue se registrar, fazer login e alterar seu perfil corretamente.



Testes Funcionais: **Teste de Aceitação**

Validam se o software atende aos requisitos do **cliente ou usuário final**.

Exemplo: **Um cliente** testando se um aplicativo de e-commerce permite adicionar e comprar produtos sem problemas.



Testes Funcionais: **Testes de Regressão**

Garantem que novas alterações ou correções **não afetem funcionalidades já existentes**.

Exemplo: Após corrigir um bug na tela de login, testar se o cadastro de usuários ainda funciona corretamente.



Testes Funcionais: **Teste de Interface**

Verificam se a interface gráfica do usuário exibe corretamente os elementos e **responde corretamente às interações**.

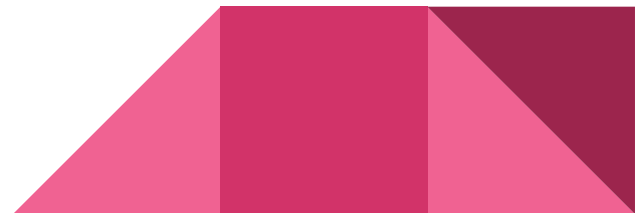
Exemplo: Conferir se um botão de envio de formulário está funcionando corretamente.



Testes Funcionais

Ferramentas para Testes Funcionais

1. Selenium – Automação de testes para aplicações web.



Testes Funcionais

Ferramentas para Testes Funcionais

1. Selenium – Automação de testes para aplicações web.
2. Postman – Testes de APIs REST.



Testes Funcionais

Ferramentas para Testes Funcionais

1. Selenium – Automação de testes para aplicações web.
2. Postman – Testes de APIs REST.
3. JUnit / Pytest – Testes de unidade e integração para Java e Python.



Testes Funcionais

Ferramentas para Testes Funcionais

1. Selenium – Automação de testes para aplicações web.
2. Postman – Testes de APIs REST.
3. JUnit / Pytest – Testes de unidade e integração para Java e Python.
4. Cypress – Testes end-to-end para aplicações web.



Teste de Unidade

Exemplo 1

Vamos supor que temos uma função chamada *calcular_media*, que recebe uma lista de números e retorna a média. Vamos escrever um teste de unidade para verificar se essa função retorna os valores corretos.



Calcular Média

```
def calcular_media(numeros):  
    if not numeros:  
        return 0 # Evita divisão por zero  
    return sum(numeros) / len(numeros)
```



Calcular Média

```
def calcular_media(numeros):  
    if not numeros:  
        return 0 # Evita divisão por zero  
    return sum(numeros) / len(numeros)
```

Possíveis cenários:

1. Testar com lista de números positivos
 2. Testar com lista de números negativos
 3. Testar com lista de números pos. e neg.
 4. Testar com lista vazia
- 

Calcular Média

```
def calcular_media(numeros):  
    if not numeros:  
        return 0 # Evita divisão por zero  
    return sum(numeros) / len(numeros)
```

Possíveis cenários:

1. Testar com lista de números positivos: [10, 20, 30]
 2. Testar com lista de números negativos: [-10, -20, -30]
 3. Testar com lista de números pos. e neg.: [-10, 0, 10]
 4. Testar com lista vazia: []
- 

Calcular Média

```
def calcular_media(numeros):  
    if not numeros:  
        return 0 # Evita divisão por zero  
    return sum(numeros) / len(numeros)
```

Possíveis cenários:

1. Testar com lista de números positivos: [10, 20, 30] -> 20
2. Testar com lista de números negativos: [-10, -20, -30] -> -20
3. Testar com lista de números pos. e neg.: [-10, 0, 10] -> 0
4. Testar com lista vazia: [] -> 0

Exemplo 2

Agora, vamos testar uma função que converte uma temperatura em Celsius para Fahrenheit.



Conversor °C para °F

```
def celsius_para_fahrenheit(celsius):  
    """Converte temperatura de Celsius para Fahrenheit."""  
    if not isinstance(celsius, (int, float)):  
        raise ValueError("A temperatura deve ser um número.")  
    return (celsius * 9/5) + 32
```



Conversor °C para °F

```
def celsius_para_fahrenheit(celsius):  
    """Converte temperatura de Celsius para Fahrenheit."""  
    if not isinstance(celsius, (int, float)):  
        raise ValueError("A temperatura deve ser um número.")  
    return (celsius * 9/5) + 32
```

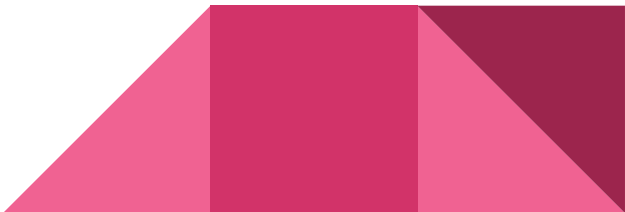
Possíveis cenários:

1. Testar com temperatura positiva inteira
 2. Testar com temperatura negativa inteira
 3. Testar com temperatura nula (zero)
 4. Testar com entrada inválida
 5. Testar com temperatura decimal
- 

Conversor °C para °F

```
def celsius_para_fahrenheit(celsius):  
    """Converte temperatura de Celsius para Fahrenheit."""  
    if not isinstance(celsius, (int, float)):  
        raise ValueError("A temperatura deve ser um número.")  
    return (celsius * 9/5) + 32
```

Possíveis cenários:

1. Testar com temperatura positiva inteira: 25
 2. Testar com temperatura negativa inteira: -5
 3. Testar com temperatura nula (zero): 0
 4. Testar com entrada inválida: "dez"
 5. Testar com temperatura decimal: 37.5
- 

Conversor °C para °F

```
def celsius_para_fahrenheit(celsius):  
    """Converte temperatura de Celsius para Fahrenheit."""  
    if not isinstance(celsius, (int, float)):  
        raise ValueError("A temperatura deve ser um número.")  
    return (celsius * 9/5) + 32
```

Possíveis cenários:

1. Testar com temperatura positiva inteira: 25 -> 77
2. Testar com temperatura negativa inteira: -5 -> 23
3. Testar com temperatura nula (zero): 0 -> 32
4. Testar com entrada inválida: "dez" -> *Entrada inválida (string)*
5. Testar com temperatura decimal: 37.5 -> 99.5

Exemplo 3

Função de Cálculo de Imposto de Renda

Agora, vamos testar uma função que calcula o imposto de renda com base na renda anual de uma pessoa. O cálculo será simplificado e pode ser baseado em faixas de renda, como segue abaixo:

1. Renda **até** R\$ 22.847,76: **isento**
 2. Renda **de** R\$ 22.847,77 **até** R\$ 33.919,80: **7,5%** de imposto
 3. Renda **de** R\$ 33.919,81 **até** R\$ 45.012,60: **15%** de imposto
 4. Renda **de** R\$ 45.012,61 **até** R\$ 55.976,16: **22,5%** de imposto
 5. Renda **acima de** R\$ 55.976,16: **27,5%** de imposto
- 

Calcular Imposto de Renda

```
def calcular_imposto(renda_anual):  
    """Calcula o imposto de renda baseado na renda anual."""  
    if renda_anual <= 22847.76:  
        return 0  
    elif renda_anual <= 33919.80:  
        return renda_anual * 0.075  
    elif renda_anual <= 45012.60:  
        return renda_anual * 0.15  
    elif renda_anual <= 55976.16:  
        return renda_anual * 0.225  
    else:  
        return renda_anual * 0.275
```

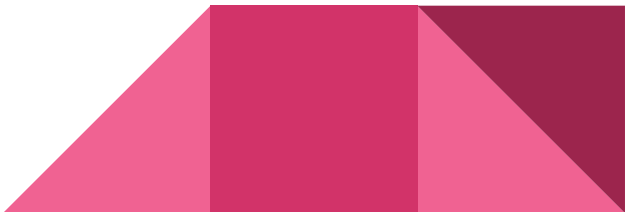
Calcular Imposto de Renda

Possíveis cenários:

1. Verifica se a pessoa com renda **até** R\$ 22.847,76 não paga imposto (faixa isenta).
 2. Testa a aplicação de 7,5% de imposto para uma renda **entre** R\$ 22.847,77 e R\$ 33.919,80.
 3. Testa a aplicação de 15% de imposto para uma renda **entre** R\$ 33.919,81 e R\$ 45.012,60.
 4. Testa a aplicação de 22,5% de imposto para uma renda **entre** R\$ 45.012,61 e R\$ 55.976,16.
 5. Testa a aplicação de 27,5% de imposto para uma renda **acima de** R\$ 55.976,16.
 6. Verifica se uma renda **de R\$ 0** resulta em imposto zero.
- 

Calcular Imposto de Renda

Possíveis cenários (**periféricos**):

1. Renda **até** R\$ **22.847,76** não paga imposto (faixa isenta).
 2. 7,5% de imposto para uma renda **entre** R\$ **22.847,77** e R\$ **33.919,80**.
 3. 15% de imposto para uma renda **entre** R\$ **33.919,81** e R\$ **45.012,60**.
 4. 22,5% de imposto para uma renda **entre** R\$ **45.012,61** e R\$ **55.976,16**.
 5. 27,5% de imposto para uma renda **acima de** R\$ **55.976,16**.
 6. Verifica se uma renda **de R\$ 0** resulta em imposto zero.
- 

Contato

davimagal.sc@gmail.com

